



GRUP 1

**2026**

**DOCUMENTACIÓ  
TÈCNICA – ES  
XUXEMONS**

Prepared By :

**Grup 1**

Prepared For :

**Carlos.M, Marc.C, Eric.G**

# Índice

|                                                               |           |
|---------------------------------------------------------------|-----------|
| <b>Descripción general y objetivos.....</b>                   | <b>3</b>  |
| Objetivos principales.....                                    | 3         |
| <b>Arquitectura de la aplicación.....</b>                     | <b>4</b>  |
| Interfaz de usuario (Angular SPA).....                        | 4         |
| Backend (API de Laravel).....                                 | 4         |
| Base de dades (MySQL).....                                    | 4         |
| <b>Diagrama de contenedores (Docker).....</b>                 | <b>5</b>  |
| Serveis Docker.....                                           | 5         |
| Volúmenes.....                                                | 5         |
| <b>Estructura de carpetas.....</b>                            | <b>6</b>  |
| <b>Tecnologías utilizadas y justificación.....</b>            | <b>8</b>  |
| Interfaz.....                                                 | 8         |
| Backend.....                                                  | 8         |
| Infraestructura.....                                          | 9         |
| <b>Requisitos para el despliegue.....</b>                     | <b>10</b> |
| Requisitos de hardware mínimos.....                           | 10        |
| Requisitos de software.....                                   | 10        |
| Puertos necesarios libres.....                                | 10        |
| <b>Instalación y puesta en marcha en local.....</b>           | <b>11</b> |
| Paso 1 – Clonar el repositorio.....                           | 11        |
| Paso 2 – Configurar las variables de entorno del backend..... | 11        |
| Paso 3 – Levantar el entorno con Docker Compose.....          | 11        |
| Paso 4 – Acceder a la aplicación.....                         | 12        |
| Paso 5 - Detener el entorno.....                              | 12        |
| Paso 6 - Reiniciar con datos limpios.....                     | 12        |
| <b>Variables de entorno.....</b>                              | <b>12</b> |
| Variables críticas.....                                       | 13        |
| Variables de base de dades.....                               | 13        |
| Variables de sesión y caché.....                              | 13        |
| <b>API – Resumen puntos finales.....</b>                      | <b>14</b> |
| Rutas públicas (sin autenticación).....                       | 14        |
| Rutas protegidas (requieren Authorization).....               | 14        |
| Rutas de administrador (rol admin).....                       | 15        |
| <b>Relaciones y tablas principales.....</b>                   | <b>17</b> |
| Usuarios de Taula.....                                        | 18        |
| Mesa chuxemones.....                                          | 19        |
| Tabla xuxedex.....                                            | 19        |
| Mesa chuches.....                                             | 20        |
| Tabla inventario.....                                         | 20        |
| Taula amigos.....                                             | 21        |
| Mesa peticiones_amistad.....                                  | 21        |
| Configuración del sistema Taula.....                          | 22        |
| Recompensas diarias de Taula.....                             | 22        |

# Descripción general y objetivos

**Xuxemons** es una aplicación web de colección y gestión de criaturas virtuales, construida como una **SPA** en Angular con backend **Laravel 12 (API REST)** y base de datos **MySQL 8.0**. Los jugadores se registran, amplían su colección e interactúan con las criaturas a diario.

El ciclo de juego se basa en la recompensa diaria: cada acceso crea un registro en **recompensas diarias** que puede otorgar nuevos Xuxemons o elementos. Cada criatura tiene un tipo (**Aigua, Terra o Aire**) y una mida (**Pequeño, Medio o Grande**), que determinan sus evoluciones según el catálogo.

La mecánica principal es alimentarlos con Xuxes (ítems apilables) para acumular experiencia en xuxedex. Al llegar al umbral **chuches\_por\_subir**, el Xuxemon evoluciona. Sin embargo, sobrealimentar puede activar el estado de **enfermedad = true**, que requiere una vacuna para curarse.

El sistema social se basa en **peticiones\_amistad** (con estados pendiente, aceptado o rechazado) y amigos para relaciones confirmadas, con búsqueda de jugadores por id.

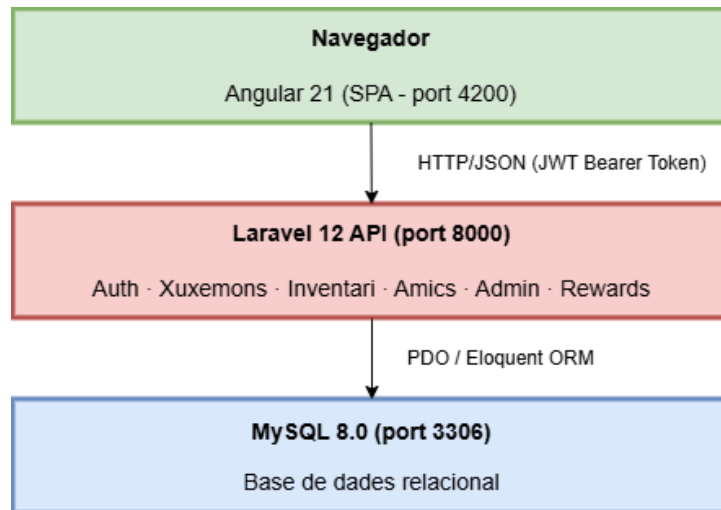
Los usuarios con rol admin tienen acceso a un panel con endpoints protegidos para gestionar cuentas, modificar Xuxedex, añadir ítems y configurar el sistema mediante **configuración del sistema**.

## Objetivos principales

- Ofrece un sistema de **colección progresiva** de Xuxemons con tres tipos de elementos (**Agua, Tierra, Aire**) y tres tamaños (**Pequeño, Medio, Grande**).
- Implementar una **Xuxedex** personal que muestra el estado de cada Xuxemon: nivel, enfermedades, posibilidad de evolución.
- Gestionar un **inventario** con dos tipos de objetos: apilables (**Suches para alimentar**) y no apilables (**vacunas para curar**).
- Incluir un sistema de **recompensa diaria** para fomentar el acceso diario.
- Disponer de un **panel de administración** para gestionar usuarios, configuración global y parámetros de cada niño.
- Permitir la **búsqueda y gestión de amistades** entre jugadores

# Arquitectura de la aplicación

La aplicación sigue una arquitectura de **tres capas separadas** que se comunican vía API REST:



## Interfaz de usuario (Angular SPA)

Aplicación **Aplicación de una sola página** que se ejecuta completamente en el navegador. La comunicación con el backend se realiza mediante **Cliente HTTP**, con un interceptor JWT que añade automáticamente el token a todas las peticiones. El acceso a las rutas está controlado con guards (**authGuard**, **noAuthGuard**, **adminGuard**) y la gestión del estado se hace de manera reactiva con **Sujeto de comportamiento**, sin librerías externas.

## Backend (API de Laravel)

API REST stateless desarrollada con Laravel 12. La autenticación se basa en JWT (**tymon/jwt-auth**), generando un token en el login que se envía en cada petición protegida. Incluye un middleware **administrador** para restringir funcionalidades de administración. Cada recurso se gestiona con su controlador y modelo Eloquent correspondiente.

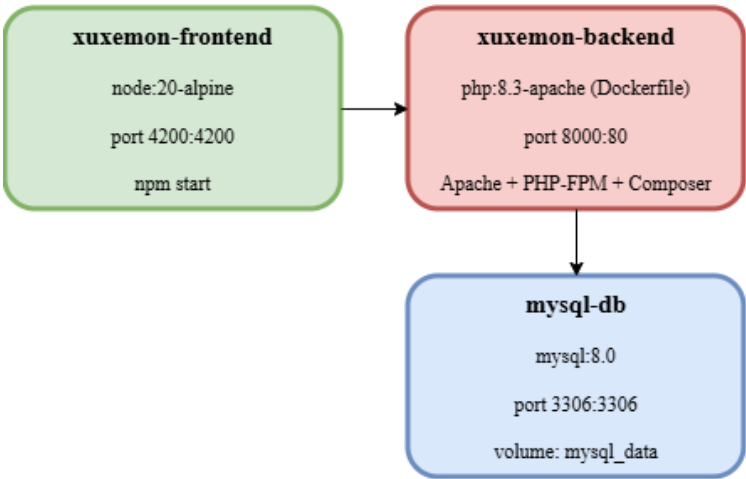
## Base de dades (MySQL)

Base de datos relacional **MySQL 8.0**. La estructura se define con migraciones de Laravel para garantizar consistencia y reproducibilidad, y los datos iniciales se cargan mediante seeders (usuarios, Xuxemons, inventario y vacunas).

# Diagrama de contenedores (Docker)

El entorno de desarrollo completo se levanta con un único **docker-compose up**.

**Docker network: red (bridge)**



## Serveis Docker

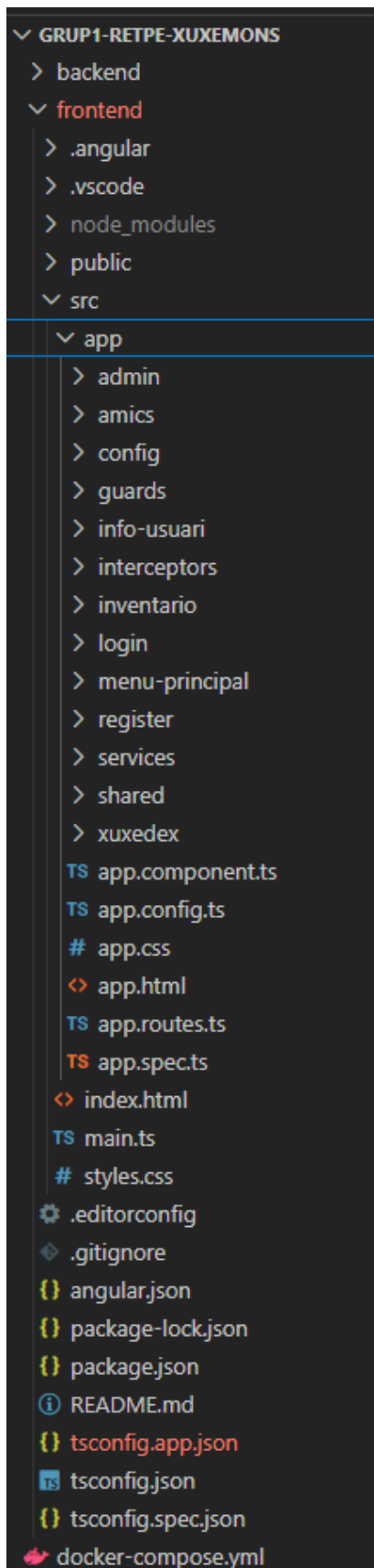
| Servicio         | Imagen         | Puerto | Descripción                    |
|------------------|----------------|--------|--------------------------------|
| xuxemon-frontend | node:20-alpine | 4200   | Servidor de desarrollo Angular |
| xuxemon-backend  | php:8.3-apache | 8000   | API Laravel sobre Apache       |
| mysql-db         | mysql:8.0      | 3306   | Base de datos relacional       |

## Volúmenes

| Servicio        | Descripción                                                |
|-----------------|------------------------------------------------------------|
| mysql_data      | Persiste los datos de MySQL entre reinicios del contenedor |
| xuxemon-backend | Montaje en caliente del código fuente del frontend         |
| mysql-db        | Montaje en caliente del código fuente del backend          |

# Estructura de carpetas

El proyecto sigue una arquitectura backend/frontend separada.



El **backend** está desarrollado con **Laravel (PHP)** i se organiza de la siguiente manera:

**Controladores:** gestionan las peticiones HTTP. Cada controlador se encarga de un ámbito concreto: autenticación, Xuxemons, inventario, amistades, recompensas diarias, vacunas, usuarios y configuración de administrador.

**Middleware:** capa de seguridad que intercepta las peticiones. Middleware de administración asegura que sólo los administradores puedan acceder a las rutas protegidas

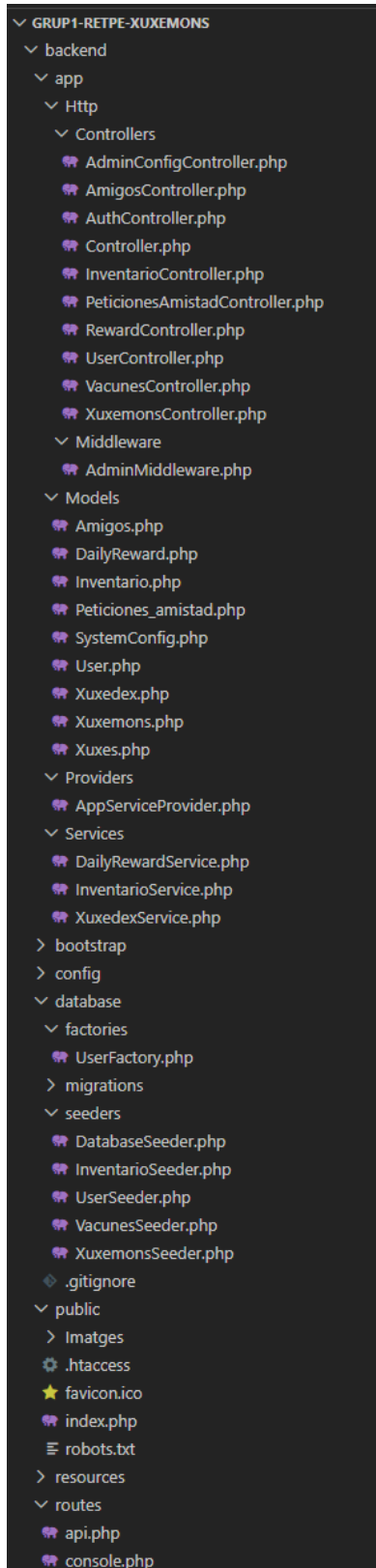
**Modelos:** representan a las entidades de la base de datos y sus relaciones: User, Xuxemons, Xuxedex, Xuxes, Inventario, Amigos, Recompensa diaria y Configuración del sistema.

**Servicios:** contienen la lógica de negocio separada de los controladores para mantener el código limpio y reutilizable.

**Base de datos:** incluye las migraciones, los seeders para poblar la base de datos con datos iniciales y las factorías para entornos de test.

El **frontend** está desarrollado con **Angular (TypeScript)** y contiene los componentes visuales, servicios de comunicación con la API, guardas de protección de rutas y los estilos de la interfaz de usuario.

El **Interfaz** está desarrollado con **Angular (TypeScript)** y todo el código fuente está en `src/app`, organizado por características:



**administrador:** gestión de usuarios y configuración accesible sólo para administradores.

**amigos:** sistema de amistades: lista de amigos y peticiones pendientes.

**configuración:** configuración general de la aplicación (interceptores HTTP, constantes, etc.).

**guardias:** protección de rutas, impidiendo el acceso a páginas sin autenticación o sin permisos de administrador.

**Información del usuario:** perfil e información personal del usuario.

**interceptores:** interceptan las peticiones HTTP para añadir automáticamente el token JWT.

**inventario:** vista y gestión de los Xuxemons que tiene cada usuario.

**Iniciar sesión / Registrarse:** formularios de autenticación y registro.

**menú-principal:** pantalla principal de la aplicación después del inicio de sesión.

**servicios:** servicios reutilizables de comunicación con la API del backend.

**compartido:** componentes y utilidades compartidas entre varias partes de la app.

**xuxedex:** vista de la colección completa de Xuxemons disponibles en el juego.

# Tecnologías utilizadas y justificación

## Interfaz

| Tecnología     | Versión | Justificación                                                                                              |
|----------------|---------|------------------------------------------------------------------------------------------------------------|
| Angular        | 21      | Framework SPA robusto con sistema de módulos, DI, routing y formularios reactivos integrados.              |
| TypeScript     | 5.x     | Tipado estático que reduce errores en tiempo de compilación y mejora la mantenibilidad.                    |
| RxJS           | 7.8     | Gestión de flujos asíncronos (HTTP, eventos) con operadores como switchMap, debounceTime, BehaviorSubject. |
| Angular Router | 21      | Navegación declarativa con guardas para proteger rutas privadas y de administrador.                        |
| HTML5 / CSS3   | -       | Maquetación semántica y estilos personalizados por componente sin dependencia de frameworks CSS externos.  |

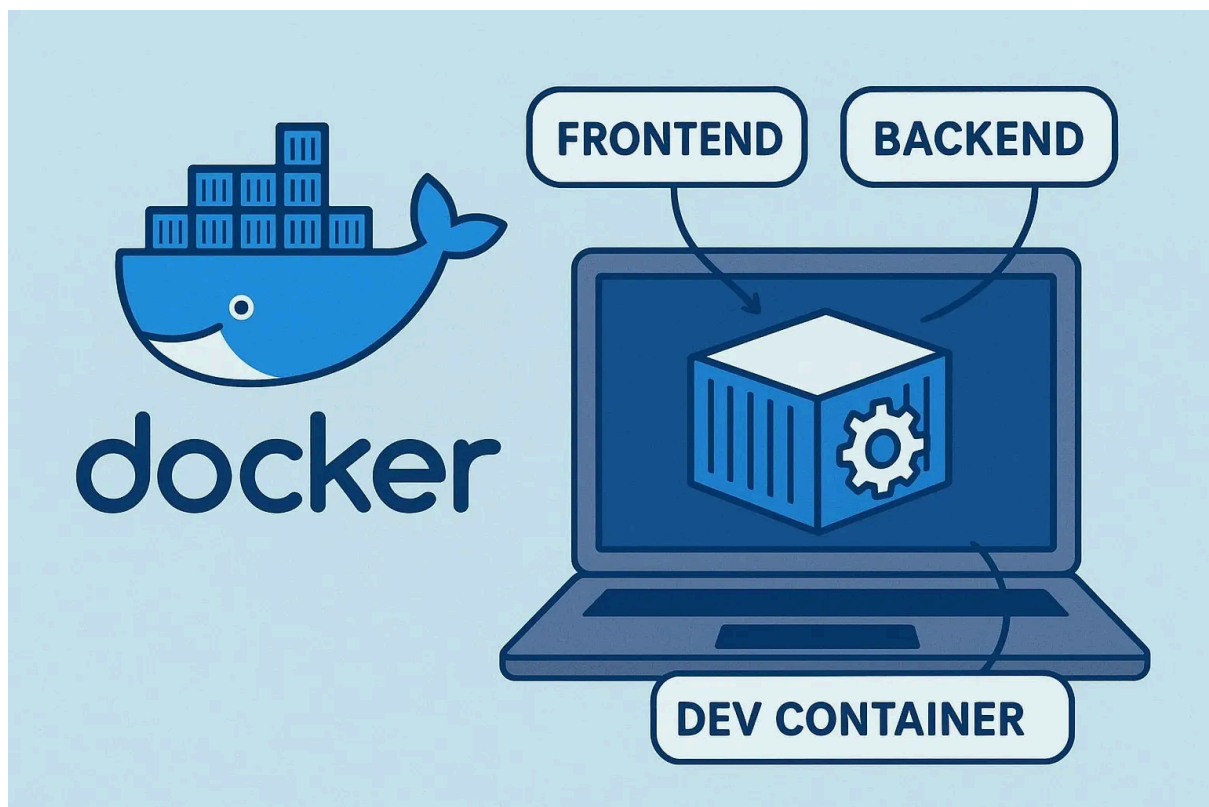
## Backend

| Tecnología | Versión | Justificación                                                                                        |
|------------|---------|------------------------------------------------------------------------------------------------------|
| PHP        | 8.3     | Lenguaje maduro para aplicaciones web, requerido por Laravel.                                        |
| Laravel    | 12      | Framework MVC con ORM (Eloquent), migraciones, sistema de middleware y enrutamiento expresivo.       |
| JWT Auth   | 2.2     | Autenticación stateless vía tokens, adecuada para APIs consumidas por SPAs.                          |
| MySQL      | 8.0     | Base de datos relacional estable y de rendimiento probado, con buen soporte de Eloquent.             |
| Apache     | 2.4     | Servidor web incluido en la imagen PHP oficial. `mod_rewrite` habilitado para el routing de Laravel. |

|          |        |                                      |
|----------|--------|--------------------------------------|
| Composer | latest | Gestor de dependencias PHP estándar. |
|----------|--------|--------------------------------------|

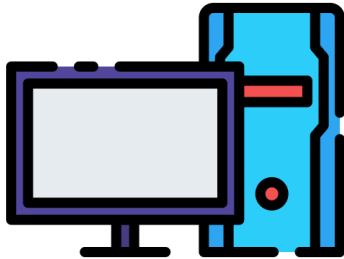
## Infraestructura

| Tecnología     | Versión        | Justificación                                                                                            |
|----------------|----------------|----------------------------------------------------------------------------------------------------------|
| Docker         | 20+            | Contenedores que garantizan que el entorno es idéntico en cualquier máquina, sin instalaciones manuales. |
| Docker Compose | v2             | Orquestación de los tres servicios (frontend, backend, BD) con un único archivo de configuración.        |
| Node.js        | 20<br>(Alpine) | Entorno de ejecución para el servidor de desarrollo de Angular y `npm`.                                  |



# Requisitos para el despliegue

## Requisitos de hardware mínimos



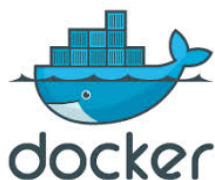
**UPC:** 2 núcleos

**RAM:** 4 GB (recomendado 8 GB para correr los tres contenedores cómodamente)

**DESCT:** 5 GB libres (imágenes Docker + código + BD)

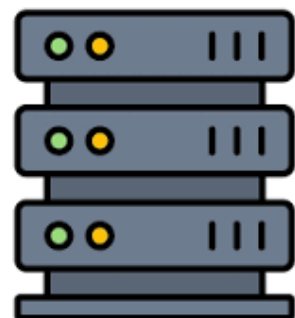
## Requisitos de software

| Software       | Versión mínima       | Notas                                                                                                                    |
|----------------|----------------------|--------------------------------------------------------------------------------------------------------------------------|
| Motor Docker   | 20.10                | Instalación oficial<br>( <a href="https://docs.docker.com/engine/install/">https://docs.docker.com/engine/install/</a> ) |
| Docker Compose | 2.0<br>(complemento) | Incluido en Docker Desktop.<br>En Linux: apt install docker-compose-plugin                                               |
| Git            | -                    | Para clonar el repositorio                                                                                               |



## Puertos necesarios libres

| Puerto | Servicio                                       |
|--------|------------------------------------------------|
| 4200   | Desarrollo frontend con Angular                |
| 8000   | Backend Laravel / API                          |
| 3306   | MySQL (opcional, para clientes de BD externos) |



# Instalación y puesta en marcha en local

## Paso 1 – Clonar el repositorio

```
$ git clone <url-del-repositorio>  
$ cd Grupo 1-Retpe-Xuxemons
```

\***Por ejemplo:** <https://github.com/Yosoyina/Grup1-Retpe-Xuxemons.git>

## Paso 2 – Configurar las variables de entorno del backend

```
$ cp backend/.env.example backend/.env
```

Editar **backend/.env** y ajustar los valores que hay en la sección **Variables de entorno**. Para un entorno local con Docker, los valores predeterminados ya funcionan.

## Paso 3 – Levantar el entorno con Docker Compose

```
$ docker compose up --build
```

### Este pedido:

1. Construye la imagen del backend (**php:8.3-apache+extensiones PHP+Compositor**).
2. Arranca MySQL y espera que esté sano (**chequeo de salud**).
3. Instala las dependencias PHP (**Composer install**).
4. Genera la clau JWT (**php artisan jwt:secret**).
5. Ejecuta las migraciones (**php artisan migrate**).
6. Carga los seeders con datos iniciales (**php artisan db:seed**).
7. Inicia **apache** todo el backend.
8. Instala las dependencias **Node.js** y arranca el servidor de desarrollo Angular.

💡 El primer arranque puede tardar 3-5 minutos mientras se descargan las imágenes y se instalan las dependencias.

## Paso 4 – Acceder a la aplicación

### URL de Angular (Frontend)



http://localhost:4200

### API REST de URL



http://localhost:4200

## Paso 5 - Detener el entorno

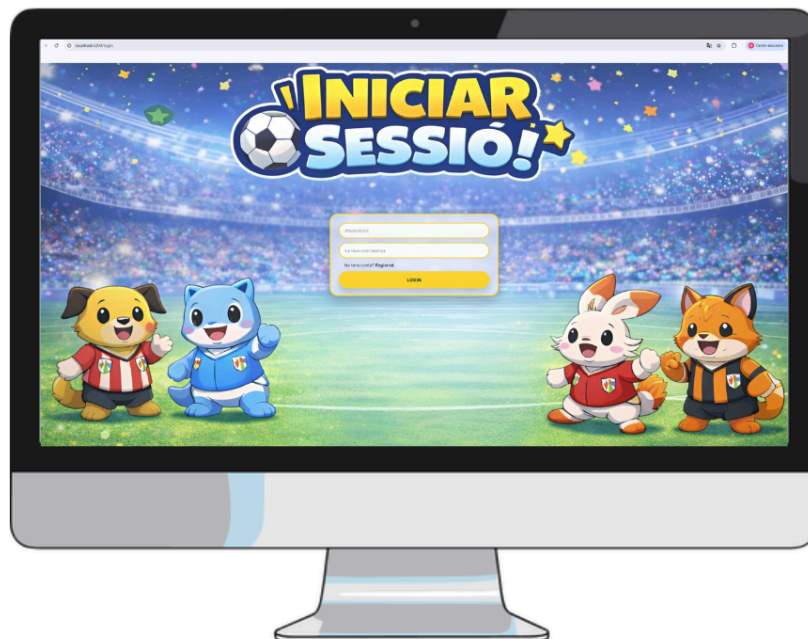
```
$ docker compose down
```

Para detener y eliminar los datos de la base de datos (**volumen**):

```
$ docker compose down -v
```

## Paso 6 - Reiniciar con datos limpios

```
$ docker compose down -v  
$ docker compose up --build
```



# Variables de entorno

Las variables de entorno del backend se definen en el archivo **backend/.env**. En entorno Docker, algunas de ellas se pueden sobrescribir directamente desde **docker-compose.yml** (la sección **ambiente** del servicio **backend** tiene prioridad).

## Variables críticas

| Variable   | Valor predeterminado (Docker)   | Descripción                                                                      |
|------------|---------------------------------|----------------------------------------------------------------------------------|
| APP_KEY    | buit – es genera automàticament | Clave de cifrado de Laravel. Se genera con <b>php artisan key:generate</b> .     |
| APP_ENV    | local                           | Entorno de ejecución. Cambiar la producción en despliegue real.                  |
| APP_DEBUG  | true                            | Activa los mensajes de error detallados. <b>Desactivar en producción.</b>        |
| APP_URL    | http://localhost                | URL base de la aplicación.                                                       |
| JWT_SECRET | generat per jwt:secret          | Clave secreta para firmar los tokens JWT. Se genera automáticamente al arrancar. |

## Variables de base de datos



```
DB_CONNECTION=mysql          # Driver de connexió
DB_HOST=127.0.0.1            # Nom del servei Docker
DB_PORT=3306                  # Port MySQL
DB_DATABASE=xuxemonsbd       # Nom de la base de dades
DB_USERNAME=dev_user         # Usuari de la base de dades
DB_PASSWORD=1234              # Contrasenya de la base de dades
```

## Variables de sesión y caché

| Variable         | Valor por defecto | Descripción                             |
|------------------|-------------------|-----------------------------------------|
| SESSION_DRIVER   | database          | Controlador de almacenamiento de sesión |
| SESSION_LIFETIME | 120               | Duración de la sesión en minutos        |
| CACHE_STORE      | database          | Controlador de caché                    |

💡 El archivo **.env** nunca debe subirse al repositorio. Está incluido en el **.gitignore** del backend.

# API – Resumen puntos finales

## Rutas públicas (sin autenticación)

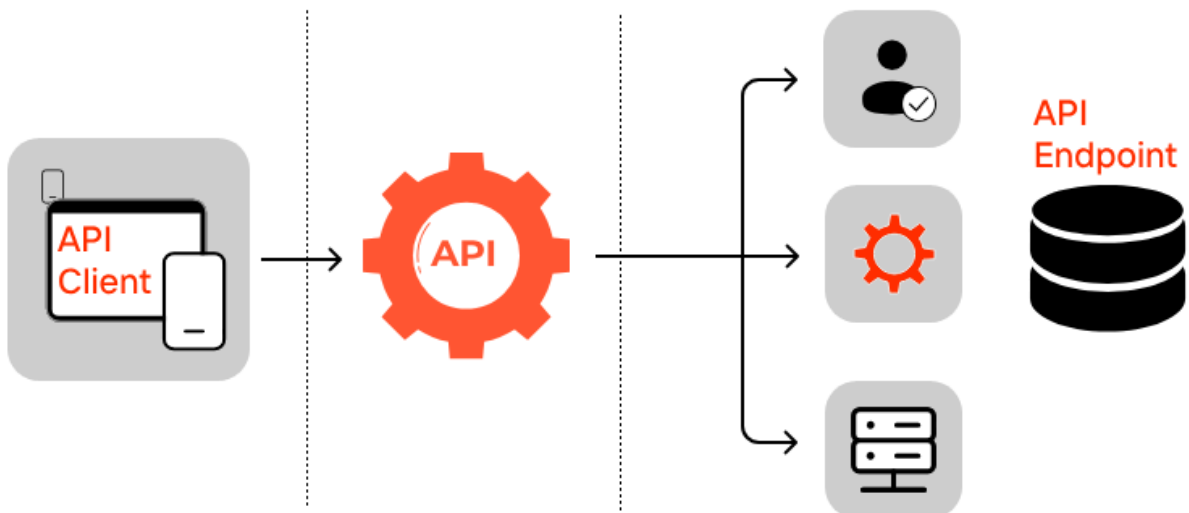
| Método | Punto final    | Descripción                     |
|--------|----------------|---------------------------------|
| CORREO | /api/registrar | Registro de un nuevo usuario    |
| CORREO | /api/login     | Login y obtención del token JWT |

## Rutas protegidas (requieren Authorization)

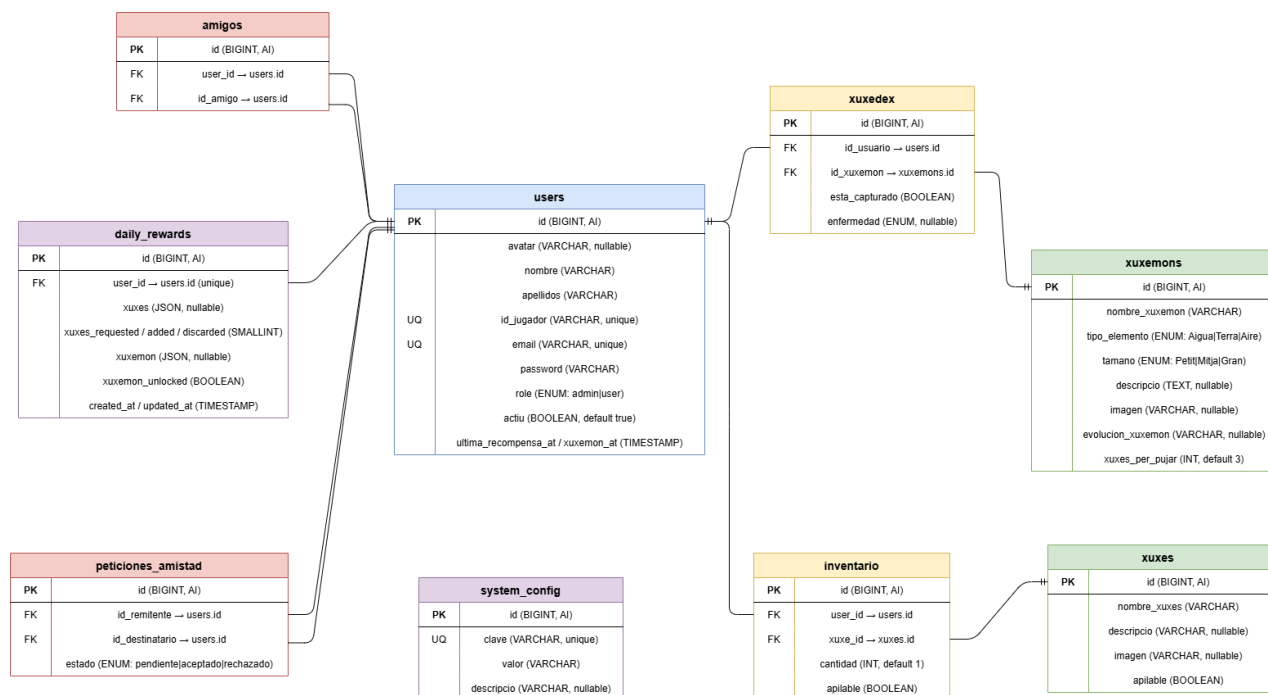
| Método | Punto final                        | Descripción                                |
|--------|------------------------------------|--------------------------------------------|
| POST   | /api/cerrar_de_sesión              | Cerrar sesión                              |
| GET    | /api/perfil                        | Obtener perfil del usuario autenticado     |
| PUT    | /api/perfil                        | Actualizar perfil                          |
| DELETE | /api/perfil                        | Eliminar cuenta                            |
| POST   | /api/recompensa                    | Reclamar la recompensa diaria              |
| GET    | /api/users/search?q=               | Buscar usuarios por ID de jugador          |
| GET    | /api/xuxedex                       | Obtener la Xuxedex del usuario autenticado |
| GET    | /api/xuxemons/{id}/evoluciones     | Cadena de evoluciones de un Xuxemon        |
| POST   | /api/xuxemons/{id}/evolucionar     | Evolucionar un Xuxemon                     |
| POST   | /api/xuxemons/{id}/feed            | Alimentar un Xuxemon                       |
| POST   | /api/xuxemons/{id}/curar           | Curar a un Xuxemon enfermo                 |
| GET    | /api/inventario                    | Obtener el inventario del jugador          |
| POST   | /api/vacunas/aplicar               | Aplicar una vacuna a un Xuxemon            |
| POST   | /api/amigos/petición               | Enviar solicitud de amistad                |
| POST   | /api/amigos/petición/{id}/aceptar  | Aceptar solicitud                          |
| POST   | /api/amigos/petición/{id}/rechazar | Rechazar solicitud                         |
| GET    | /api/amigos                        | Listado de amigos                          |
| GET    | /api/amigos/peticiones-pendientes  | Solicitudes recibidas pendientes           |
| GET    | /api/amigos/peticiones-enviadas    | Solicitudes enviadas pendientes            |
| DELETE | /api/amigos/{friendId}             | Eliminar un amigo                          |

## Rutas de administrador (rol admin)

| Método | Punto final                       | Descripción                                |
|--------|-----------------------------------|--------------------------------------------|
| GET    | /api/admin/users                  | Listar a todos los usuarios                |
| PUT    | /api/admin/users/{id}/toggle      | Activar / desactivar cuenta                |
| PUT    | /api/admin/users/{id}/toggle-role | Cambiar rol (user ↔ admin)                 |
| GET    | /api/admin/xuxedex                | Xuxedex de un usuario específico           |
| POST   | /api/admin/xuxedex                | Añadir Xuxemon aleatorio a un usuario      |
| GET    | /api/admin/inventory/items        | Listar todos los elementos disponibles     |
| POST   | /api/admin/inventario             | Añadir item al inventario de un usuario    |
| GET    | /api/admin/config                 | Conseguir configuración global del sistema |
| PUT    | /api/admin/config/{clave}         | Actualizar un parámetro de configuración   |
| GET    | /api/admin/xuxemons-nivel         | Xuxemons con su coste de evolución         |
| PUT    | /api/admin/xuxemons-nivel/{id}    | Actualizar coste de evolución              |



# Relaciones y tablas principales



El diagrama superior muestra el modelo entidad-relación completo de la base de datos de Xuxemons. La mesa central es **usuarios**, que actúa como núcleo del modelo y se relaciona directamente con la mayoría de las demás tablas.

Las tablas están agrupadas por color según su función: en **azul** la tabla principal de usuarios; en **verde** las tablas de catálogo del juego (**xuxemons** i **chuches**); en **amarillo** las tablas pivót que establecen las relaciones N:M (**xuxedex** i **inventario**); en **rojo/rosa** las mesas sociales (**amigos** i **peticiones\_amistad**); en **lila** las tablas de sistema (**recompensas diarias** i **configuración del sistema**).

Las líneas que unen las tablas representan las claves foráneas (FK), indicando qué tabla referencia qué otra. A continuación se describe detalladamente cada tabla con todos sus campos.

## Usuaris de Taula

Conténe tots els jugadors registrats en la aplicació. És la taula central del model.

| Acampar                      | Tipo       | Descripción                                                               |
|------------------------------|------------|---------------------------------------------------------------------------|
| id                           | BIGINT PK  | Identificador intern autoincrementat.                                     |
| avatar                       | VARCHAR    | Nombre del arxiv de imatge del avatar (per defecte avatarpordefecto.png). |
| nombre                       | VARCHAR    | Nom real del jugador.                                                     |
| apellidos                    | VARCHAR    | Apellidos del jugador.                                                    |
| id_jugador                   | VARCHAR UQ | Nombre de usuari únic visible en el joc.                                  |
| email                        | VARCHAR UQ | Correu electrònic, usat per el login.                                     |
| password                     | VARCHAR    | Contraseña cifrada (bcrypt).                                              |
| role                         | ENUM       | Rol de usuari: administrador o usuari.                                    |
| actiu                        | BOOLEAN    | Indica si la taula està activa (true) o ha sigut desactivada per admin.   |
| ultima_recompensa_at         | TIMESTAMP  | Moment en el que se reclamó la última recompensa de Xuxes.                |
| ultima_recompensa_xuxemon_at | TIMESTAMP  | Moment en el que se reclamó la última recompensa de Xuxemon.              |
| created_at / updated_at      | TIMESTAMP  | Marcas de tiempo de Laravel.                                              |

## Mesa chuxemones

Catálogo global de todas las criaturas del juego. Cada fila representa un tipo de Xuxemon.

| Acampar                 | Tipo      | Descripción                                                           |
|-------------------------|-----------|-----------------------------------------------------------------------|
| id                      | BIGINT PK | Identificador del Xuxemon.                                            |
| nombre_xuxemon          | VARCHAR   | Nombre de la criatura.                                                |
| tipo_elemento           | ENUM      | Elemento al que pertenece: Agua, Tierra o Aire.                       |
| tamaño                  | ENUM      | Tamaño de la criatura: Pequeño, Mediano o Grande.                     |
| descripcio              | TEXT      | Descripción libre de la criatura (opcional).                          |
| imagen                  | VARCHAR   | Nombre del archivo de imagen (opcional).                              |
| evolucio_xuxemon        | VARCHAR   | Nombre del Xuxemon al que evoluciona (null si es forma final).        |
| xuxes_per_pujar         | INT       | Número de Xuxes necesarias para evolucionar (o para la forma Grande). |
| created_at / updated_at | TIMESTAMP | Marcas de tiempo de Laravel.                                          |

## Tabla xuxedex

Mesa pívot que registra qué Xuxemons ha capturado a cada jugador y su estado actual.

 **Restricción:** el parell (id\_usuario, id\_xuxemon) es único.

| Acampar                 | Tipo      | Descripción                                                                     |
|-------------------------|-----------|---------------------------------------------------------------------------------|
| id                      | BIGINT PK | Identificador del registro.                                                     |
| id_usuario              | BIGINT FK | Referencia al jugador (users.id).                                               |
| id_xuxemon              | BIGINT FK | Referencia a Xuxemon (xuxemons.id).                                             |
| esta_capturado          | BOOLEAN   | true si el jugador lo tiene en su colección.                                    |
| enfermedad              | ENUM      | Enfermedad activa: Intoxicación por azúcar, sobredosis, Atracón o nulo (todos). |
| created_at / updated_at | TIMESTAMP | Marcas de tiempo de Laravel.                                                    |

## Mesa chuches

Catálogo de objetos disponibles en el juego: alimentos (apilables) y vacunas (no apilables).

| Acampar                 | Tipo      | Descripción                                              |
|-------------------------|-----------|----------------------------------------------------------|
| id                      | BIGINT PK | Identificador del objeto.                                |
| nombre_xuxes            | VARCHAR   | Nombre del objeto.                                       |
| descripción             | VARCHAR   | Breve descripción (opcional).                            |
| imagen                  | VARCHAR   | Nombre del archivo de imagen (opcional).                 |
| apilable                | BOOLEAN   | true → alimento acumulable; false → vacuna de uso único. |
| created_at / updated_at | TIMESTAMP | Marcas de tiempo de Laravel.                             |

## Tabla inventario

Mesa pívot que relaciona jugadores con los objetos que poseen, indicando la cantidad.

| Acampar                 | Tipo      | Descripción                                                             |
|-------------------------|-----------|-------------------------------------------------------------------------|
| id                      | BIGINT PK | Identificador del registro.                                             |
| user_id                 | BIGINT FK | Jugador propietario (users.id).                                         |
| xuxe_id                 | BIGINT FK | Objeto del inventario (chuches.id).                                     |
| cantidad                | INT       | Número de unidades que tiene el jugador (relevante para los apilables). |
| apilable                | BOOLEAN   | Copia del campo apilable de chuches para optimizar consultas.           |
| created_at / updated_at | TIMESTAMP | Marcas de tiempo de Laravel.                                            |

## Taula amigors

Registra las relaciones de amistad confirmadas entre dos jugadores. Cada amistad genera dos filas (una por cada sentido).

 **Restricción:** el par (user\_id, ID de amigo) es único.

| Acampar                 | Tipo      | Descripción                            |
|-------------------------|-----------|----------------------------------------|
| id                      | BIGINT PK | Identificador del registro.            |
| user_id                 | BIGINT FK | Jugador que tiene el amigo (users.id). |
| id_amigo                | BIGINT FK | Jugador amigo (users.id).              |
| created_at / updated_at | TIMESTAMP | Marcas de tiempo de Laravel.           |

## Mesa peticiones\_amistad

Gestiona las solicitudes de amistad enviadas entre jugadores y su estado.

 **Restricción:** el par (id\_remitente, ID del destinatario) es único.

| Acampar                 | Tipo      | Descripción                                    |
|-------------------------|-----------|------------------------------------------------|
| id                      | BIGINT PK | Identificador del registro.                    |
| id_remitente            | BIGINT FK | Jugador que envía la sol·licitud (users.id).   |
| id_destinatario         | BIGINT FK | Jugador que rep la sol·licitud (users.id).     |
| estado                  | ENUM      | Estat actual: pendiente, aceptado o rechazado. |
| created_at / updated_at | TIMESTAMP | Marcas de tiempo de Laravel.                   |

## Configuración del sistema Taula

Tabla de configuración global del sistema editable desde el panel de administración. Funciona con pares clave-valor.

 **Parámetros predefinidos:**chuches\_hora\_recompensa, chuches\_cantidad\_diaria, xuxemon\_hora\_recompensa, infeccio\_bajon, entre otros.

| Acampar                 | Tipo       | Descripción                                               |
|-------------------------|------------|-----------------------------------------------------------|
| id                      | BIGINT PK  | Identificador del registro.                               |
| clave                   | VARCHAR UQ | Nombre único del parámetro (ej: chuches_cantidad_diaria). |
| valor                   | VARCHAR    | Valor actual del parámetro.                               |
| descripción             | VARCHAR    | Explicación del parámetro (opcional).                     |
| created_at / updated_at | TIMESTAMP  | Marcas de tiempo de Laravel.                              |

## Recompensas diarias de Taula

Registra el estado de la recompensa diaria de cada jugador. Tiene un único registro por jugador que se actualiza todos los días.

| Acampar                 | Tipo         | Descripción                                                                     |
|-------------------------|--------------|---------------------------------------------------------------------------------|
| id                      | BIGINT PK    | Identificador del registro.                                                     |
| user_id                 | BIGINT FK UQ | Jugador propietario (users.id). Un registro único por jugador.                  |
| xuxes                   | JSON         | Detalle de las chuches otorgadas en la última recompensa.                       |
| xuxes_requested         | SMALLINT     | Chuches solicitadas en la última recompensa.                                    |
| xuxes_added             | SMALLINT     | Chuches efectivamente añadidas al inventario.                                   |
| xuxes_discarded         | SMALLINT     | Chuches descartados (inventario lleno).                                         |
| xuxemon                 | JSON         | Datos del Xuxemon otorgado (si los hubiere).                                    |
| xuxemon_unlocked        | BOOLEAN      | true si Xuxemon se desbloqueó en la última recompensa.                          |
| created_at / updated_at | TIMESTAMP    | Timestamps de Laravel (el updated_at indica quan es va reclamar la recompensa). |